

Bases de Datos I

PROCEDIMIENTOS ALMACENADOS (STORED PROCEDURE) - FUNCIONES

Rutinas Almacenadas

- Cuando múltiples aplicaciones cliente se escriben en distintos lenguajes o funcionan en distintas plataformas, pero necesitan realizar la misma operación en la base de datos.
- Cuando la seguridad es muy importante, proporciona un entorno seguro y consistente donde las aplicaciones y los usuarios no obtendrían ningún acceso directo a las tablas de la base de datos, sólo pueden ejecutar algunos procedimientos almacenados.



Rutinas Almacenadas

- Conjunto de sentencias disponibles para las aplicaciones que acceden a un RDBMS.
- Almacenados en el Gestor de la Bases de Datos.
- Por defecto, se crea en la base de datos actual. Si queremos crearla en una BD distinta, deberemos hacer preceder el nombre de la BD de destino.
- El delimitador de línea de comando por defecto (;) necesita ser cambiado a la hora de codificar rutinas almacenadas.

Rutinas Almacenadas

- Cuando se invocan, se realiza un **USE DBName** implícito (y se deshace cuando la rutina termina).
- Cuando se borra una DB, todas las SR almacenadas en él, se borran también.
- MySQL soporta 3 tipos de rutinas:
 - 1.Procedimientos Almacenados (SP),
 - 2.Funciones Almacenadas (SF),
 - 3.Triggers o Disparadores.



Rutinas Almacenadas

- Tiene un nombre
- Puede tener una lista de parámetros
- Puede contener un conjunto de sentencias SQL.
- Se crean utilizando el comando **CREATE**.

Que es un procedimiento almacenado?

- Un procedimiento almacenado es un conjunto de comandos e instrucciones que pueden almacenarse en el servidor.

Cuando es conveniente utilizarlos?

- Cuando múltiples aplicaciones cliente necesitan realizar la misma operación en la base de datos.
- Cuando la seguridad es muy importante. Se crean procedimientos para realizar operaciones comunes.

Ventajas:

- Los procedimientos almacenados pueden mejorar el rendimiento ya que se necesita enviar menos información entre el servidor y el cliente.
- Proporciona un entorno seguro y consistente. En este entorno, las aplicaciones y los usuarios no obtendrían ningún acceso directo a las tablas de la base de datos, sólo pueden ejecutar algunos procedimientos almacenados.
- **Compatibilidad:** los procedimientos almacenados MySQL permiten ejecutar la misma operación sobre la base de datos también en el caso de que las aplicaciones cliente estén implementadas en distintas plataformas y programadas en distintos lenguajes.
- **Integridad:** gracias a los procedimientos almacenados MySQL es posible centralizar el acceso a la información.
- **Seguridad:** cuando necesitamos evitar el acceso directo a la base de datos, los procedimientos almacenados MySQL permiten establecer un entorno seguro, otorgando permisos y privilegios para la ejecución.

- **Rendimiento:** con los procedimientos almacenados MySQL todo el trabajo se ejecuta en el servidor, por lo cual se minimiza de forma importante la cantidad de información intercambiada con las aplicaciones cliente y se reduce el tráfico de acceso a la base de datos además del número de accesos.
- **Centralización:** los procedimientos almacenados MySQL permiten centralizar la lógica comercial ofreciendo a todas las aplicaciones clientes la misma versión actualizada, lo que hace el mantenimiento más sencillo.
- **Reutilización del código:** al escribir un mismo código que se ejecuta por todas las aplicaciones, es posible reducir las inconsistencias.

Tipos de parámetros

- **IN.**
 - Es el parámetro por defecto. Puede cambiar su valor dentro de la rutina, pero, permanece inalterable su valor externo.
- **OUT.**
 - En el momento de la llamada, su valor es NULL, pero, puede ser modificado dentro de la rutina y dicho valor estará disponible en el exterior de la rutina.
- **INOUT.**
 - Se comporta como una combinación de los 2 anteriores. Se le puede asignar un valor en la llamada, que puede ser modificado en el interior y estar disponible en el exterior.



Procedimientos Almacenados

- Aunque no tenga parámetros, hay que finalizar la definición de la cabecera con ().
- No obstante, un SP sin parámetros, puede ser invocado sin utilizar los paréntesis.
- Se le invoca utilizando el comando **CALL**.
- Un SP puede mostrar resultados o devolver los resultados en las variables OUTPUT especificadas.



Sintaxis de un procedimiento

```
CREATE PROCEDURE nombre_procedimiento ( parametros )  
BEGIN  
    Instrucciones  
END
```

Llamada a un procedimiento

```
Call nombre_proc(parametros)
```

Parámetros

Parámetros IN: son la entrada a las funciones y procedimientos

Parámetros OUT: permite comunicar valores al bloque de programa que invocó al subprograma

Procedimientos Almacenados

```
CREATE PROCEDURE mayor_menor (IN x INT)
BEGIN
    IF (x > 5) THEN
        SELECT CONCAT(x, " es mayor") as resp;
    ELSE
        SELECT CONCAT(x, " es menor ") as resp;
    END IF;
END
```



Parámetros (tipos)

IN	OUT	IN OUT
es el tipo por defecto	debe ser especificado	debe ser especificado
pasa valores a un subprograma	retorna valores a quien lo llamó	pasa valores iniciales al subprograma y retorna un valor actualizado a quien lo llamó
un parámetro formal actúa como una constante	parámetros formales actúan como variables	parámetros formales actúan como una variable inicializada
a un parámetro formal no se le puede asignar un valor	a un parámetro formal debe asignársele un valor	a un parámetro formal podría asignársele un valor
los parámetros actuales pueden ser constantes, variables inicializadas, literales o expresiones	los parámetros actuales deben ser variables	los parámetros actuales deben ser variables

EJEMPLO 1

EL PROCEDIMIENTO CUENTA_EMP CUENTA LOS REGISTROS QUE TIENE LA TABLA EMPLEADOS Y ALMACENA EL RESULTADO EN LA VARIABLE VAR1.

RECUERDE QUE ANTES DE CREAR EL PROCEDIMIENTO HAY QUE UBICARSE EN LA BASE DE DATOS CORRESPONDIENTE. EN ESTE CASO EMPRESA.

USE EMPRESA;

-----CREAMOS EL PROCEDIMIENTO-----

delimiter //

create procedure cuenta_emp (out var1 int)

begin

select count(*) into var1 from empleado;

end//

delimiter ;

-----LLAMAMOS EL PROCEDIMIENTO-----

call cuenta_emp(@var1);

Tambien puede utilizar otros nombres de variable cuando llama el procedimiento. Recuerde que en esta se almacena el resultado del procedimiento.

Call cuenta_emp(@contar);

-----**VER EL CONTENIDO DE LA VARIABLE**-----

```
select @var1;
```

o si utilizo la variable contar:

```
select @contar;
```

-----**USAR EL PROCEDIMIENTO**-----

Si vamos a hacer una consulta que nos muestre el promedio del salario de los empleados teniendo en cuenta el procedimiento `cuenta_emp` que nos proporciona la cantidad total de empleados almacenado en la variable `var1`:

```
Select sum(salario)/@var1 "PROMEDIO" from empleado;
```

Procedimiento con parámetros de entrada y de salida

DELIMITER //

```
CREATE PROCEDURE ObtenerNombreApellidoSalida(idAlumno INT, OUT nombreApellido  
VARCHAR(200))
```

BEGIN

```
SELECT CONCAT(nombre, ' ', apellido) INTO nombreApellido
```

```
FROM estudiantes WHERE id = idAlumno;
```

```
END //
```

DELIMITER ;

Al llamar a este nuevo procedimiento, no es necesario crear la variable @nombre.

```
CALL ObtenerNombreApellidoSalida(2, @nombreApellido);
```

```
SELECT @nombreApellido
```

-- Solución 1. Utilizando SET

```
DELIMITER $$ DROP PROCEDURE IF EXISTS contar_productos$$  
CREATE PROCEDURE contar_productos(IN gama VARCHAR(50), OUT total INT UNSIGNED)  
BEGIN SET total = ( SELECT COUNT(*) FROM producto WHERE producto.gama = gama);  
END $$ DELIMITER ;  
CALL contar_productos('Herramientas', @total); SELECT @total;
```

-- Solución 2. Utilizando SELECT ... INTO

```
DELIMITER $$ DROP PROCEDURE IF EXISTS contar_productos$$  
CREATE PROCEDURE contar_productos(IN gama VARCHAR(50), OUT total INT UNSIGNED)  
BEGIN SELECT COUNT(*) INTO total FROM producto WHERE producto.gama = gama;  
END $$  
DELIMITER ; CALL contar_productos('Herramientas', @total); SELECT @total;
```

Funciones Almacenadas

- Una SF se crea utilizando el comando **CREATE** .
- Los parámetros de entrada (IN) no son requeridos en una SF.
- Una SF debe tener una sentencia RETURN y sólo puede devolver un valor.

Funciones Almacenadas

- Una SF es llamada simplemente con su nombre (sin usar CALL).
- El nombre de una SF debe ser diferente a la de cualquier función SQL.
- La diferencia entre una función y un procedimiento es que la función devuelve valores. Estos valores pueden ser utilizados como argumentos para instrucciones SQL, tal como lo hacemos con otras funciones como: MAX() o COUNT().

SINTAXIS

DELIMITER //

```
CREATE FUNCTION nombre_funcion(parametro1 TIPO_DATO, parámetro N TIPO_DATO )  
RETURNS TIPO_DATO_RETORNO  
BEGIN  
declaracion y asignacion de varianles  
sentencias sql  
END//
```

DELIMITER ;

CREATE FUNCTION nombre_funcion(parametro1 TIPO_DATO, parametroN TIPO_DATO): define el nombre de la función y los parámetros que recibirá con sus tipos de datos correspondientes.

RETURNS: indica el tipo de dato que tendrá el valor de salida de la función

BEGIN: determina el inicio de las sentencias que podemos realizar en la función

END: determina el fin de las sentencias

DELIMITER //: es un delimitador que nos permite indicarle al motor de MySQL sobre donde

Funciones Almacenadas

```
CREATE FUNCTION es_mayor(x INT) RETURNS CHAR(3)
BEGIN
  IF (x > 5) THEN
    RETURN 'SI';
  ELSE
    RETURN 'NO';
  END IF;
END
```

- Utilizar la cláusula **RETURNS** es obligatorio al momento de definir una función y sirve para especificar el tipo de dato devuelto (sólo el tipo de dato, no el dato).

```
Delimiter //
CREATE FUNCTION cuadrado (s SMALLINT) RETURNS SMALLINT
Begin
  RETURN s*s;
End //
Delimiter ;
```



Funciones Almacenadas

DELIMITER //

```
CREATE FUNCTION saludar(nombre VARCHAR(50), apellido VARCHAR(50), email VARCHAR(100)) RETURNS  
VARCHAR(150)  
BEGIN
```

```
DECLARE nombre_completo VARCHAR(100);  
SET nombre_completo = CONCAT(nombre, ' ', apellido);  
IF email IS NOT NULL THEN  
    RETURN CONCAT('Bienvenido ', nombre_completo);  
ELSE  
    RETURN 'Aun no tenes Email';  
END IF;  
END//
```

DELIMITER;

```
SELECT id, nombre, apellido, email, saludar(nombre, apellido, email) AS 'Saludo' FROM  
empleados;
```

```
delimiter //  
create function divide(dividendo int,divisor int) returns int  
begin
```

```
declare aux int;  
declare contador int;  
declare resto int;  
set contador = 0;  
set aux = dividendo;
```

```
while aux >= divisor do  
    set aux = aux- divisor;  
    set contador = contador + 1;  
end while;  
set resto = dividendo - aux ;  
return contador;
```

```
End //  
Delimiter ;
```

QUE ES UN TRIGGER?

- Técnicamente, un trigger es un objeto de una base de datos que está asociado a una tabla, y que será activado cuando la acción que tiene asociada tiene lugar.
- El trigger se puede ejecutar cuando tiene lugar una acción INSERT, UPDATE o DELETE, siendo posible su ejecución tanto antes como después del evento.
- Está construido por bloques PL/SQL y sentencias SQL.

Ámbitos de aplicación de triggers:

- **AUDITORIA**

Los triggers se usan para llenar tablas de auditoría, en donde se registren ciertos tipos de transacciones o accesos hacia tablas.

- **REGLAS DE NEGOCIO**

Cuando ciertas reglas de negocio son muy elaboradoras y necesitan sean expresadas a nivel base de datos, es necesario que además de reglas y restricciones se utilicen triggers.

- **VALIDACIÓN DE DATOS**

Los triggers pueden controlar ciertas acciones, por ejemplo: pueden rechazar o modificar ciertos valores que no cumplan determinadas reglas, para prevenir que datos inválidos sean insertados en la base.

SEGURIDAD

- Refuerzan las reglas de seguridad de una aplicación.
- Integridad de los datos mejorada.
- Aseguran que las operaciones relacionadas se realizan juntas de forma implícita.
- Ofrece un mayor control sobre la Base de Datos.

DESVENTAJAS

- Hay que programarlos para cada DBMS.
- Un Trigger nunca se llama directamente.
- Por funcionalidad, no hay que poner en uno solo las funciones de INSERT, UPDATE y DELETE.
- Utilizar moderadamente los triggers.

Sintaxis de un trigger

```
DELIMITER //
```

```
CREATE TRIGGER nombre_trigger Condición Evento ON nombre_tabla FOR  
EACH ROW  
BEGIN  
    Instrucciones SQL  
END;  
DELIMITER ;
```

Evento:

Se especifica el comando DML de SQL que dispara el Trigger: **Delete, Insert o Update**

Condición:

BEFORE, antes de ejecutar el evento.

AFTER, después de ejecutar el evento

Instrucciones :

Se definen mediante comandos y sentencias SQL y PL/SQL, la tarea específica que realiza el Trigger.



Las columnas de una tabla asociada con un disparador pueden referenciarse empleando las variables OLD y NEW

Variable NEW

Con esta variable podemos acceder a los datos introducidos. Con NEW.nombre_columna se almacenará la información con el nuevo valor que tendrá ese registro modificado (desde un UPDATE o INSERT) en la tabla.

Los trigger relacionados con DELETE no tendrán disponible la variable NEW.

Variable OLD

OLD a diferencia de NEW, almacena el valor de las columnas que van a ser borradas o eliminadas.

Al igual que pasa con NEW, OLD no está disponible en todas las instrucciones, más concretamente el valor no se puede recuperar cuando la instrucción es un INSERT.

- Ejemplo. Compara si el salario ha aumentado en más del 25% de salario antiguo:

:NEW.salario > (:OLD.salario*1.25)

COMPROBACION DE DATOS

```
DELIMITER //  
CREATE TRIGGER comprobar_edad AFTER INSERT ON cliente FOR  
EACH ROW  
BEGIN  
  IF NEW.edad < 0 THEN  
    SET NEW.edad = 0;  
  END IF;  
END;  
DELIMITER ;
```

COMPROBACION DE DATOS

```
DELIMITER //
```

```
CREATE TRIGGER actualizarPrecioProducto BEFORE UPDATE ON  
productos FOR EACH ROW
```

```
BEGIN
```

```
IF NEW.coste <> OLD.coste THEN
```

```
    SET NEW.precio = NEW.coste * 2;
```

```
END IF ;
```

```
END //
```

```
DELIMITER ;
```

HISTORICO DE PRODUCTOS

```
DELIMITER //  
CREATE TRIGGER trigger_producto_historico AFTER INSERT ON producto FOR EACH  
ROW  
BEGIN  
INSERT INTO producto_historico(id, descripcion, precio)  
VALUES (NEW.id, NEW.descripcion, NEW.precio, CURDATE());  
END;  
DELIMITER ;
```